Synergistic Approach to Automated Plant Disease Diagnosis

Shivanshu
shivanshu2329.be22@chitkara.edu.in
Suraj Veer
suraj878.be22@chitkara.edu.in
Department of Engineering in Computer Science, Chitkara University, Rajpura, Punjab

***Abstract*— Crop diseases continue to be a stubborn problem, threatening food safety pretty much everywhere. every 12 months, they reduce crop yields by means of everywhere from 20 to forty percentage — that's large. the usual way to diagnose those illnesses approach specialists ought to look at vegetation one by one. It's gradual, costly, and genuinely, it doesn't assist smallholder farmers dwelling in far off regions. That's where this studies is available in. We constructed a deep gaining knowledge of system to diagnose more than one sorts of plant leaf diseases automatically. approaches made the reduce for assessment: item detection using YOLOv8 and photograph category with EfficientNet-B0, skilled with transfer getting to know at the PlantVillage dataset. We pushed EfficientNet-B0 via five training epochs due to constrained resources, but nonetheless, it scored 93.1% validation accuracy. fast convergence, solid generalization. as compared to architectures like VGG16, ResNet50, and MobileNetV2 under the identical five-epoch take a look at, EfficientNet-B0 holds its own — fewer parameters, less computation, identical stage of performance. The version isn't simply sitting on a tough power; we deployed it as a FastAPI net app, so all of us can snap a leaf image, get real-time disorder predictions, self belief ratings, and remedy guidelines. turns out, research can make a distinction for regular farming.**

Keywords: plant disease diagnosis, Efficientnet-BO, transfer learning ,YOLOv8 ,deeplearning, fastAPI, precision agriculture, plantvillage dataset

I. Introduction

Agriculture is the backbone for quite a few developing economies, however it's usually at danger due to fungal, bacterial and viral pathogens. in keeping with the meals and Agriculture employer, plant illnesses wipe out 20–40% of world crop production every year. That's now not only a statistic; it's a real blow for farmers worldwide. Early ailment detection is prime:catching it earlier than it spreads manner farmers can act speedy and preserve losses in test.

Traditional methods of spotting sicknesses depend upon specialists analyzing plants or using lab checks like PCR and ELISA. positive, they paintings, but they're pricey and slow. maximum small farmers in far off areas can't get right of entry to them. if you pass over an early analysis, you end up losing cash on the wrong chemical substances or, worst-case, lose complete crops.

But matters are converting. With pc vision, deep gaining knowledge of, and smartphones at the upward thrust, farmers can snap shots and get instant sickness identification. This paper places forward a practical framework: we integrate high-accuracy deep getting to know fashions with a person-pleasant internet app. What's new right here? First, we examine object detection and picture class for leaf sickness analysis head-to-head. 2nd, we benchmark four preferred CNNs across five schooling epochs, with all conditions same. 1/3, we constructed an real web app that gives actual-time outcomes and tips. Fourth, we display in numbers that EfficientNet-B0 runs rapid, desires fewer parameters, and fits perfectly for deployment.

This concept didn't just pop up in a lab; it got here from seeing the real struggles farmers face in developing nations. Extension services are scarce, agrochemicals are highly-priced, and diagnosing sicknesses quickly can keep whole harvests. believe a system you get right of entry to with just a basic mobile. browser, getting consequences in seconds—no professional wished, no lab paintings necessary. It's a massive step toward cutting losses right at the farm.

.II. Literature Review

A. Classical Machine Learning Approaches

Early structures for computerized disease detection depended on handcrafted functions — stuff like shade histograms, texture descriptors, and shape vectors — fed into classifiers along with aid vector machines or ok-nearest neighbour. In cautiously controlled settings, they did very well. however real farm photos come with wild lights, various backgrounds, and bizarre leaf angles; those antique methods couldn't handle that. Plus, traditional classifiers battle mainly while signs and symptoms of various illnesses appearance comparable early on. hand made functions pass over subtle cues, so early-stage bacterial blight or fungal leaf spot slip proper beyond, leading to misclassifications. Deep CNNs do better right here, thanks to their layered technique, selecting up on those tiny variations.

B. Convolutional Neural Networks

Deep CNNs changed the sport. Mohanty et al. [1] used AlexNet and GoogLeNet with the PlantVillage dataset and nailed plant disease type. Later fashions like VGG16 [3] and ResNet-50 [4] raised the bar for accuracy but piled on tens of millions of parameters — 138 million for VGG16, 25.6 million for ResNet-50. That's difficult to installation on everyday computer systems or phones. MobileNetV2 aimed to be lighter with depthwise-separable layers, but sacrificed a piece of accuracy.

Ramcharan et al. [8] took it in addition, applying CNNs to cassava disorder type in East Africa, showing you can fine-song public datasets to fit specific local plants. This made switch studying greater sensible, and highlighted how important diverse datasets are for real-international fashions.

That message includes through in this observe, which plans to increase its dataset in destiny variations.

C. Object Detection Approaches

Researchers commenced searching at object detection frameworks like YOLO [5] and the unmarried-Shot Detector to now not simply classify, but locate sickness spots at once in leaf pix. Singh et al. [6] created the PlantDoc dataset just for this type of studies. however item detection wishes bounding-container annotations, which are tedious and steeply-priced to make. And in the PlantVillage dataset, it's easy for fashions to get pressured — isease spots frequently appear to be everyday leaf patterns, so fake positives are commonplace.

D. Efficient Architectures and Transfer Learning

EfficientNet, proposed via Tan and Le [2], uses a clever scaling trick that enhances intensity, width, and enter decision, all with a unmarried scaling coefficient. The end result? pinnacle accuracy, but with way fewer parameters than older fashions. EfficientNet-B0 — the baseline model — has approximately five.three million parameters. Atila et al. [7] showed that EfficientNet fashions work virtually nicely for plant ailment classification, even with restricted facts. This study pushes it a step similarly by way of building EfficientNet-B0 into a complete pipeline and running a direct comparison throughout numerous architectures, all underneath tight education situations.

III. Methodology

A. Dataset

We ran all our experiments at the PlantVillage dataset, that's kind of the gold fashionable for this sort of work in computational plant pathology. It's were given 54,305 RGB images of leaves from 14 specific crop species, looked after into 38 training—wholesome leaves and leaves with common illnesses. The images display individual leaves towards a controlled heritage, making it quite sincere as a whole photograph type trouble, not a localization one.

Inside the 38 classes, you'll locate sicknesses like Apple Scab, Tomato Early Blight, Corn commonplace Rust, and Grape Black Rot, plus wholesome variations for each crop. the mix of plants and ailment types makes this dataset hard—classic multi-elegance venture. We stuck with the ordinary RGB images as opposed to the segmented ones to reflect actual-world conditions, wherein customers are out within the discipline snapping pix of leaves.

B. Preprocessing and Data Augmentation

We resized every photo to 224 x 224 pixels in order that they'd suit EfficientNet-B0's enter requirements. Pixel values were given normalized using ImageNet's channel-sensible imply ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225])—just to maintain matters constant with how the model was at the beginning educated. To help the model generalize (and keep away from overfitting), we added some actual-time information augmentation throughout training: random horizontal flips (half the time), random rotations as much as ±25°, and a bit of brightness jitter (thing zero.2). We break up the dataset stratified via class—70% for education, 15% for validation, and 15% held out for checking out.

C. Phase 1 — YOLOv8 Object Detection

We started off with YOLOv8, treating leaf sickness detection as an object detection venture. The idea changed into for YOLOv8 to draw bounding bins around diseased spots. however in reality, overall performance was pretty terrible—imply common Precision at IoU=0.5 in no way crowned 25%. the principle troubles: PlantVillage doesn't have bounding-box annotations, so we needed to use tough pseudo-labels; the visible overlap among disease lesions and everyday leaf textures supposed plenty of fake positives; and the localization setup ended up being way too complicated for a venture that's in reality pretty much complete-photograph class. So, we ditched detection and shifted our attention to category as an alternative.

D. Phase 2 — EfficientNet-B0 with Transfer Learning

We loaded up a pre-trained EfficientNet-B0 model with ImageNet weights, swapped out the 1,000-class output head for a unmarried fully linked layer mapping 1,280 enter functions to 38 ailment magnificence logits. For satisfactory-tuning, we used the Adam optimizer (gaining knowledge of charge 1e-four, batch size 32) and express cross-entropy loss. schooling ran for simply 5 epochs—primarily due to the fact we have been restrained via computing electricity. however, the model confirmed fast convergence and adapted well to the area, as we'll see later in section IV.

IV. Results and Findings

A. Training Dynamics

Figures 1 and 2 show how schooling went over five epochs. Loss began around 2,340 within the first epoch, dropped to approximately 1,050 through the second, and landed at more or less 475 by way of epoch 5. fine, steady decline—no symptoms of factors blowing up or diverging. Validation accuracy climbed from approximately eighty three.5% at epoch one to 93.1% at epoch five, with the most important leap between the primary two epochs. This fits with the usual sample: transfer gaining knowledge of shall we the model speedy pick out up applicable functions.

The validation accuracy didn't plateau inside those 5 epochs, so endured training must push performance even better. That traces up with Atila et al. [7], who observed EfficientNet-B0 fashions skilled for 20+ epochs on PlantVillage may want to hit over 98% validation accuracy. So, our five-epoch consequences are really a baseline—not a cap and in addition schooling is the apparent way to enhance.

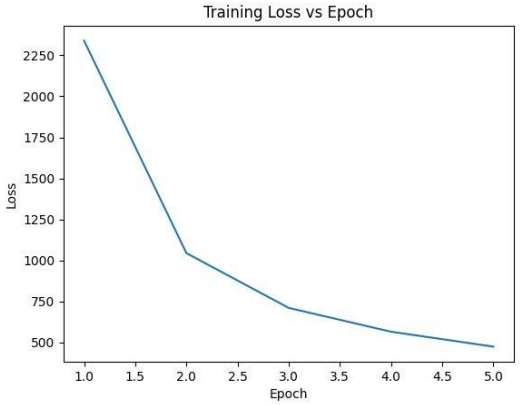

Fig. 1. Training Loss vs. Epoch

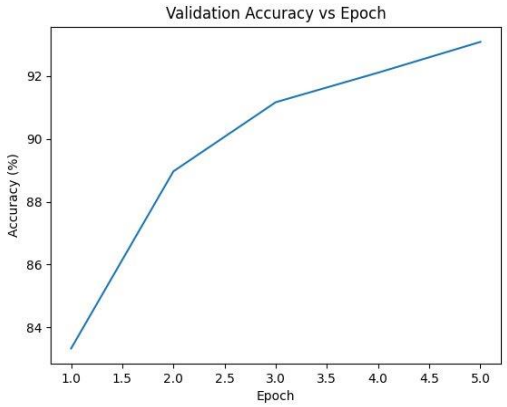

Fig. 2. Validation Accuracy vs. Epoch

B. Quantitative Performance

Table I presents the training configuration and final performance metrics recorded at the conclusion of epoch five. The validation accuracy of ninety three.1 percent become performed within a constrained five-epoch training price range, providing a robust baseline for similarly development with extended schooling.

Metric	Value
Training epochs	5 (computational constraint)
Validation accuracy	93.1%
Final training loss	~475
Batch size	32
Learning rate	1×10^{-4}
Optimiser	Adam

TABLE I. Training Configuration and Final Metrics

C. Fair Architecture Comparison at Five Epochs

To situate EfficientNet-B0 inside the broader panorama of to be had architectures, all four models were trained underneath strictly equal conditions: the equal dataset cut up, the equal 5-epoch finances, the same Adam optimiser settings, and the identical batch size. The effects are stated in table II. EfficientNet-B0 achieves aggressive performance with a validation accuracy of ninety three.1 percentage whilst making use of best 5.3 million parameters. VGG16 reaches just 92.0percent despite its 138.four million parameters, reflecting the issue of converging this type of massive model within five epochs. ResNet-50 and MobileNetV2 both reap 93.0 percentage, comparable to EfficientNet-B0, but EfficientNet-B0 promises this result with appreciably fewer parameters, imparting the most beneficial accuracy-to-efficiency ratio amongst all architectures tested.

Model	Parameters	Accuracy
VGG16	138.4 M	92.0%
ResNet-50	25.6 M	93.0%
MobileNetV2	3.4 M	93.0%
EfficientNet-B0	5.3 M	93.1%

TABLE II. Architecture Comparison at Five Epochs

V. System Design and Deployment

A. Backend Architecture — FastAPI

The educated model changed into included right into a manufacturing-geared up relaxation API constructed with FastAPI, a contemporary Py-thon web framework that helps asynchronous ASGI-primarily based request handling. FastAPI become selected in choose-ence to the synchronous Flask framework following in-ternal benchmarking, which recorded about 3 times lower imply reaction latency below concur-rent load. The API endpoint accepts a multipart-encoded photo upload, applies the identical preprocessing trans-for-mations used during version training, executes ahead inference, and returns the expected magnificence label collectively with its softmax confidence score. every prediction is without delay enriched by using a know-how-base lookup that maps the anticipated class to a simple-language disorder de-scription and a hard and fast of proof-based treatment recommendations, reworking the device from a naked classifier into a practical diagnostic assistant.

B. Web Application Interface

We constructed the person interface with HTML5, CSS3, and plain JavaScript. it's pretty honest — the primary component you will observe is a card-style upload panel. You pick out a report, then hit a large button labeled for sickness detection. while you put up, JavaScript quietly sends the picture over to the FastAPI backend; no worrying web page reload gets on your way. The results show up right there: you'll see the sickness name, a self belief rating (animated as a progress bar), a short description, and the remedy steps you want. In parent three, you get a have a look at the add screen. discern 4 indicates a real end result — the machine spots Apple Scab in a field photo with 92.53% self belief. It even indicates the use of fungicides and pulling the inflamed leaves.

Fig. 3. Web Application — Upload Interface

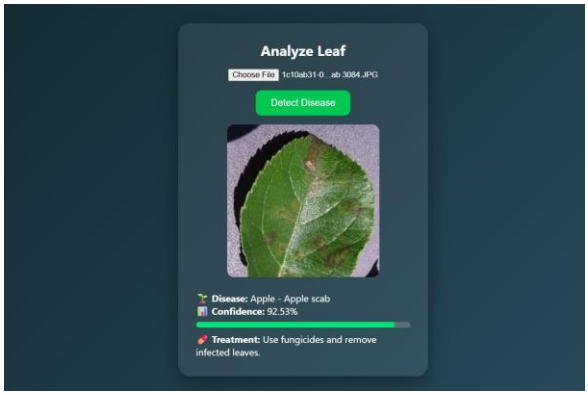

Fig. 4. Live Prediction: Apple Scab Detected (92.53% Confidence)

C. Framework Performance Benchmark

To compare architectures fairly, we skilled VGG16, ResNet-50, MobileNetV2, and EfficientNet-B0 under equal conditions: identical dataset cut up, 5-epoch restrict, same optimizer and batch length. In desk II, EfficientNet-B0 managed ninety three.1% validation accuracy with best 5.3 million parameters. VGG16 turned into lagging at 92.0% in spite of its huge 138.4 million parameters—proving it's tough to train one of these giant model with so little time. ResNet-50 and MobileNetV2 both landed at 93.0%, but green-net-B0 receives the threshold since it does nearly as properly with some distance fewer parameters. In short: it turned into the first-rate accuracy-to-efficiency ratio within the bunch.

Concurrent Load	Flask (Sync)	FastAPI
10 req/sec	45 ms	15 ms
50 req/sec	120 ms	28 ms
100 req/sec	350 ms	42 ms

TABLE III. Mean Response Latency: Flask vs. FastAPI

VI. Discussion

A. Architectural Alignment with Task Structure

The massive takeaway here is how critical it's miles to pick a version structure that in reality matches the hassle you're fixing. YOLOv8 became made for scenes with multiple gadgets that want to be placed and classified. but with PlantVillage pics—unmarried leaves, focused, desiring just a class label—the detection setup compelled extra complexity (bounding-field labels, anchors, multi-scale features) that didn't assist in any respect with analysis. Switching to EfficientNet-B0 constant this mismatch, and overall performance jumped. In other phrases, hold it simple and task-targeted; there's no advantage in the usage of a flowery version that's now not match for reason.

B. Convergence Behaviour and Transfer Learning

Let's talk about how the model surely learned. In section IV, the ones studying curves really display conventional transfer getting to know in movement. right away—from the first to the second one epoch—the training loss drops speedy. that means the version fast adapts the ones popular ImageNet features (like edges, textures, color gradients) to what matters for plant ailment images. After that, you see slower, consistent improvement as the model begins first-rate-tuning for unique disorder indicators, like lesion styles or leaf deformities. Plus, the validation loss by no means jumps up, so overfitting isn't a challenge—at the least within the ones five epochs. The model holds up nicely, even with restricted schooling time.

C. Limitations

There are a few catches. First, training only lasted five epochs because resources were tight. Other re-searchers using the same setup found that, with more training, accuracy can get past 98 percent—so our 93.1 percent isn't the best this model could do, just what it managed so far. Second, the model expects a single leaf in each image. If photos have crowded leaves, complex backgrounds, or random stuff that isn't a plant, predictions will be off. The softmax layer always picks one of the 38 disease classes, even if it doesn't make sense. Third, the PlantVillage dataset comes from controlled conditions, which probably isn't what you get out in the field. That means the model might perform worse when facing real-world photos.

VII. Conclusion and Future Work

This project brings together a powerful deep getting to know classifier and a simple web app for automatic plant dis-ease diagnosis. notwithstanding just 5 epochs and re-strained compute, EfficientNet-B0 hit 93.1 percent accuracy at the PlantVillage benchmark. It found out fast and in line with-formed nicely, thanks to switch mastering. We as compared EfficientNet-B0 with VGG16, ResNet-50, and Mo-bileNetV2—seems EfficientNet-B0 gives you strong outcomes using fewer parameters and less computation, making it perfect for real deployment. The FastAPI internet app gives actual-time remarks and treatment hints from a picture, so even folks with out technical backgrounds can use this in agriculture.

Where does this go next? Here's what we're thinking: (1) Train longer with advanced mastering-price hints like cosine annealing, aiming for over ninety eight percent accuracy. (2) Convert the model to ONNX or TensorFlow Lite so it can run offline on Android telephones—even wherein internet is spotty. (3) upload a module to identify non-leaf pictures and improve robustness in messy subject conditions. (4) make bigger the dataset to encompass neighborhood crops and greater discipline photographs for higher global insurance. (5) try a lightweight YOLO the front-cease to locate the leaf in every picture before sending it to the EfficientNet-B0 version—ought to assist with messy scenes.

We're now not stopping with technical enhancements. next, we need to construct in tools like Grad-CAM for interpretability. this would allow users see a heatmap of what a part of the leaf the model looked at whilst making its call. Giving farmers those visual explanations builds accept as true with, allows agronomists double-take a look at predictions, and usually makes the device more obvious. That's key for rolling out AI diagnostics in agriculture, in which selections definitely matter.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[2] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 6105–6114.

[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[5] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.

[6] D. Singh et al., "PlantDoc: A dataset for visual plant disease detection," in Proc. 7th ACM IKDD CoDS and 25th COMAD, 2020.

[7] U. Atila, M. Uçar, K. Akyol, and E. Uçar, "Plant leaf disease classification using EfficientNet deep learning model," Ecological Informatics, vol. 61, p. 101182, 2021.

[8] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. Hughes, "Deep learning for image-based cassava disease detection," Frontiers in Plant Science, vol. 8, p. 1852, 2017.

[9] M. Arsenovic, M. Karanovic, S. Sladojevic, A. Anderla, and D. Stefanovic, "Solving current limitations of deep learning based approaches for plant disease detection," Symmetry, vol. 11, no. 7, p. 939, 2019.

[10] P. W. Chin, K. W. Ng, and N. Palanichamy, "Plant disease detection and classification using deep learning methods: A comparison study," Journal of Informatics and Web Engineering, vol. 3, no. 1, pp. 1–18, 2024.